Trust No Input: Application-Layer Metadata Tampering in Quantum Policy Verification

—

BreaQ 2026 — Challenge 1 Technical Investigation & Defense Report

Team: $\mu - Qbit$ (DEMO-TEAM)
Participants: Richard Watry, Dylane Mahazoasy
Date: October 25, 2026

Abstract

This report presents our complete investigation, three-boundary fault localization, minimal reproducer, permutation exploit analysis, and canonical decoding defense for BreaQ 2026 Challenge 1 (*Trust No Input*). Using local ideal simulation in Qiskit, we prove that verification failures on migrated and externally imported quantum assets (`MIG-DELTA` and `INCIDENT-7`) are not caused by quantum decoherence, gate errors, or circuit construction bugs. Instead, they stem from an application-layer trust-boundary violation in `ChallengeService.execute`, which allows untrusted manifest `compatibility_map` permutations to reorder canonical quantum outputs via `compatibility_decode()` prior to authorization. We implement a canonical decoder (`secure_decode` and `secure_decision`) that restores 100% soundness and completeness across unseen fingerprints, spoofing attacks, n-bit generalizations ($n = 16$), and blind metamorphic invariance tests.

1 Executive Summary, Architecture & Threat Model

1.1 Security Claim and Root-Cause Layer

Based on our experimental triage, three-boundary inspection, and controlled counterfactual interventions, our primary security finding is:

> **Security Claim:** *The verification failures in `MIG-DELTA` and `INCIDENT-7` are caused not by quantum noise or circuit construction, but by an application-layer trust-boundary violation in `ChallengeService.execute`, which passes untrusted manifest `compatibility_map` permutations into `compatibility_decode()` prior to the authorization check; enforcing canonical $q_0 \dots q_{n-1}$ decoding (`secure_decode` / `secure_decision`) restores 100% soundness and completeness across all permutations.*

Root-Cause Layer: Classical application post-processing / reporting adapter layer (`compatibility_decode` called inside `ChallengeService.execute` in `breaq_ch1/service.py`).

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

1.2 System Architecture and Contract Analysis

The policy-fingerprint verification service encodes a 4-bit asset fingerprint across four canonical logical slots ordered as (`alpha`, `beta`, `gamma`, `delta`), where slot $i \in \{0, 1, 2, 3\}$ is mapped to Qiskit qubit q_i:

1. **Quantum Circuit Construction (`build_bv_probe`):** Constructs a 5-qubit, 4-classical-bit Bernstein-Vazirani (BV) circuit. Qubits $q_0 \dots q_3$ are initialized in $|+\rangle = H|0\rangle$, while ancilla q_4 is prepared in $|-\rangle = HX|0\rangle$. For each index i where `fingerprint`$[i]$ == `'1'`, a $\mathrm{CX}(q_i, q_4)$ gate kicks back a -1 phase onto q_i, which a final layer of Hadamard gates converts into $|1\rangle$ prior to measurement $q_i \to c_i$.

2. **Authoritative vs. Untrusted Representations:** Under the system contract, the asset registry's `registry_fingerprint` is defined in the canonical logical-slot order (q_0, q_1, q_2, q_3), and the ideal backend simulator produces authoritative raw measurement `counts`. By contrast, the manifest's `compatibility_map` crosses an external trust boundary (`migration-adapter` or `external-import`) and must be treated as **untrusted presentation metadata** that must never influence a security decision.

1.3 Warm-Up: Qiskit Endianness vs. Canonical Logical Order

Qiskit labels qubits from q_0 upward, but formats classical measurement keys in big-endian bitstring order $c_{n-1} \dots c_1 c_0$ (placing the highest-index classical bit c_{n-1} on the left):

- **2-Qubit Warm-Up:** Applying $X(0)$ to $|00\rangle$ flips q_0 to $|1\rangle$ while q_1 remains $|0\rangle$, producing the displayed Qiskit key $|q_1 q_0\rangle =$ `'01'` with probability 1.0.

- **4-Qubit Warm-Up:** Applying $X(1)$ and $X(2)$ to $|0000\rangle$ produces the displayed key $|q_3 q_2 q_1 q_0\rangle =$ `'0110'`. Reversing this string via `canonical_bits_from_counts` recovers the canonical (q_0, q_1, q_2, q_3) sequence `'0110'` ($q_0 =$ `0`, $q_1 =$ `1`, $q_2 =$ `1`, $q_3 =$ `0`). For non-palindromic states such as `MIG-DELTA` (raw key `'0101'`), reversing is mandatory to obtain the true canonical state (`'1010'`).

1.4 Threat Model

Table 1 summarizes the threat model constructed prior to source inspection.

Element	Specification
Protected Asset	Quantum policy-fingerprint verification decision (`MATCH` / `MISMATCH`) and canonical logical-slot mapping (`alpha`, `beta`, `gamma`, `delta`) $\leftrightarrow$ (q_0, q_1, q_2, q_3).
Required Property	**Soundness & Completeness:** Every authentic probe matching `registry_fingerprint` must yield `MATCH`, and every mismatched or spoofed probe must yield `MISMATCH`, independent of presentation metadata.
Adversary Capability	Manipulation of untrusted manifest metadata (`compatibility_map`) in migrated or externally imported manifests.
Trusted Inputs	`registry_fingerprint` (asset registry), `build_bv_probe` (BV circuit builder), and raw backend `counts` from the ideal simulator.
Untrusted Inputs	Manifest `compatibility_map` and the adapter output `reported_fingerprint`.
Observable Symptom	Valid migrated/imported assets (`MIG-DELTA`, `INCIDENT-7`) fail verification with `MISMATCH` despite deterministic $512/512$ backend counts.
Initial Alternative Explanations	Quantum gate decoherence, readout/shot noise, or a bit-reversal bug inside the quantum circuit builder.

Table 1: Threat model for Challenge 1 (`01_trust_no_input.ipynb`).

2 Phase A (Triage) & Phase B (Three-Boundary Fault Localization)

2.1 Phase A & B: Triage and Three-Boundary Evidence Table

We executed all four cases in `team_instance.json` (512 shots, `seed=2026`) and inspected the system across three independent boundaries:

1. **Boundary 1 — Prepared Quantum Circuit (`get_circuit` & `Statevector`):** Removing final measurements from `service.get_circuit("INCIDENT-7")` and evolving its exact `Statevector` confirms that the BV oracle contains $\mathrm{CX}(q_0, q_4)$ and $\mathrm{CX}(q_1, q_4)$, deterministically preparing $|q_3 q_2 q_1 q_0\rangle = |0011\rangle$ (canonical `'1100'`) with probability 1.0 (and 50/50 split across ancilla $q_4 \in \{|0\rangle, |1\rangle\}$ when sampled on all 5 qubits: `{'00011': 520, '10011': 504}`). The quantum circuit boundary is 100% intact.
2. **Boundary 2 — Raw Backend Output (`counts` & `canonical_bits_from_counts`):** Across all four cases, the simulator returns a single deterministic bitstring for all 512/512 shots. Converting the dominant raw key into canonical order via `canonical_bits_from_counts(counts)` yields `'1010'` for `MIG-DELTA` and `'1100'` for `INCIDENT-7`, which **exactly match** their respective `registry_fingerprint` values. Quantum noise and readout error are completely ruled out.
3. **Boundary 3 — Application Report (`compatibility_decode`):** In `ChallengeService.execute`, the canonical bitstring is permuted by `compatibility_decode(counts, case["compatibility_map"])` before comparing against `registry_fingerprint`. Because `MIG-DELTA` carries `compatibility_map = [1, 0, 2, 3]` and `INCIDENT-7` carries `[3, 1, 2, 0]`, the adapter corrupts `'1010'` → `'0110'` and `'1100'` → `'0101'`, causing false `MISMATCH` decisions.

Diagnostic Field	`CAL-ALPHA`	`CAL-BETA`	`MIG-DELTA`	`INCIDENT-7`
Provenance	`factory-cal`	`factory-cal`	`migration-adapter`	`external-import`
Manifest `compatibility_map`	`[0, 1, 2, 3]`	`[0, 1, 2, 3]`	`[1, 0, 2, 3]`	`[3, 1, 2, 0]`
Raw Backend Counts (512 shots)	`{'1001': 512}`	`{'0110': 512}`	`{'0101': 512}`	`{'0011': 512}`
Dominant Raw Key ($c_3c_2c_1c_0$)	1001	0110	0101	0011
Canonical Interpretation ($q_0q_1q_2q_3$)	1001	0110	1010	1100
Expected `registry_fingerprint`	1001	0110	1010	1100
Adapter `reported_fingerprint`	1001	0110	0110	0101
Oracle Matches Registry?	`True`	`True`	`True`	`True`
Legacy Adapter Corrupted Output?	`False`	`False`	`True`	`True`
Legacy Service Decision	`MATCH`	`MATCH`	`MISMATCH`	`MISMATCH`

Table 2: Three-boundary localization evidence across all four cases (`localization_evidence`).

2.2 Falsifiable Causal Claim and Controlled Experiment

> **Causal Claim:** *Changing `compatibility_map` from a non-identity permutation (`[1, 0, 2, 3]` or `[3, 1, 2, 0]`) to the identity permutation `[0, 1, 2, 3]` while holding the quantum circuit and registry fingerprint constant changes `reported_fingerprint` to match the canonical output and flips the verification decision from `MISMATCH` to `MATCH`.*

Using `service.simulate_adapter(target_fp, cmap)`, we held the quantum circuit fixed for `MIG-DELTA` (`'1010'`) and `INCIDENT-7` (`'1100'`) and varied only `compatibility_map`. Replacing the faulty map with `[0, 1, 2, 3]` left raw counts (`{'0101': 512}` and `{'0011': 512}`) and canonical fingerprints unchanged while restoring `reported_fingerprint` to `'1010'` and `'1100'` and flipping both decisions from `MISMATCH` to `MATCH`.

3 Phase C — Minimal Reproducer

To construct the smallest possible reproducer (`minimal_reproducer`), we selected a single-gate source fingerprint `'1000'` (Hamming weight 1, encoding a single $\mathrm{CX}(q_0, q_4)$ gate) and compared two metadata configurations containing only a single adjacent transposition of slots 0 and 1:

- **Configuration A (`config_identity`):** `compatibility_map = [0, 1, 2, 3]` $\Longrightarrow$ raw counts `{'0001': 512}`, canonical `'1000'`, reported `'1000'` $\Longrightarrow$ `MATCH`.
- **Configuration B (`config_swapped`):** `compatibility_map = [1, 0, 2, 3]` $\Longrightarrow$ raw counts `{'0001': 512}`, canonical `'1000'`, reported `'0100'` $\Longrightarrow$ `MISMATCH`.

4 Phase D — Defensive Fix & Verification Suite

4.1 Trusted Canonical Decoder Implementation

To remediate the vulnerability without removing legacy presentation helpers for UI/reporting consumers, we implemented `secure_decode` and `secure_decision`:

```
def secure_decode(counts: dict[str, int]) -> str:
    """Return the fingerprint in canonical q0...q(n-1) order."""
    if not counts:
        raise ValueError("Measurement counts dictionary cannot be empty.")
    return canonical_bits_from_counts(counts)

def secure_decision(counts: dict[str, int], registry_fingerprint: str) -> str:
    decoded_fingerprint = secure_decode(counts)
    _validate_bits(registry_fingerprint, n=len(decoded_fingerprint))
    return "MATCH" if decoded_fingerprint == registry_fingerprint else "MISMATCH"
```

By comparing `canonical_bits_from_counts(counts)` directly against the validated `registry_fingerprint`, `secure_decision` completely excludes `compatibility_map` from the security boundary.

4.2 Positive, Negative, and Unseen Spoofing Evaluation

We evaluated `secure_decision` across 13 test cases in `run_defense_tests()` (`submission.json`):

- **4 Instance Cases (Completeness):** `CAL-ALPHA`, `CAL-BETA`, `MIG-DELTA`, and `INCIDENT-7` all return `MATCH` ($4/4$ passed).
- **5 Unseen Positive Cases under Non-Identity Maps (Generalization):** Fingerprints `'0000'`, `'1111'`, `'0101'`, `'1011'`, and `'0010'` tested with non-identity permutations (`[1,0,2,3]`, `[3,2,1,0]`, `[1,0,3,2]`, `[2,3,0,1]`, `[0,2,1,3]`) all decode to the exact canonical fingerprint and return `MATCH` ($5/5$ passed).
- **4 Unseen Negative Spoofing Cases (Soundness):** An attacker with a wrong oracle circuit attempts to spoof a `MATCH` against a target `registry_fingerprint` by supplying a crafted permutation (e.g., oracle `'0101'` vs. registry `'1100'` with `[3,1,2,0]`; oracle `'0010'` vs. registry `'1000'` with `[2,1,0,3]`; oracle `'1011'` vs. registry `'1110'` with `[0,2,3,1]`; and oracle `'1111'` vs. registry `'0000'`). While the legacy adapter would falsely accept the first three as `MATCH`, `secure_decision` rejects all four with `MISMATCH` ($4/4$ passed, total $13/13$ passed, 0 failed).

5 Phase E — Exploit Objective & Hamming-Weight Invariant

5.1 Permutation Exploit Construction

When an attacker controls `compatibility_map` in the legacy service, they can transform a canonical source fingerprint $s \in \{0,1\}^n$ into any permuted report $r_j = s_{\pi(j)}$ for $\pi \in S_n$. Our exploit function `craft_compatibility_map(source, target)` constructs π in $\mathcal{O}(n)$ time:

1. Verify len(s) == len(t) and valid binary characters via `_validate_bits`.
2. Check the **Hamming-weight invariant**: if s.count(`'1'`) $\neq$ t.count(`'1'`), return `None` (unreachable).

3. Partition indices of s into queues of zero-positions $I_0 = \{i \mid s_i = \texttt{'0'}\}$ and one-positions $I_1 = \{i \mid s_i = \texttt{'1'}\}$, then iterate over each bit t_j of `target`, popping the next index from I_0 if $t_j = \texttt{'0'}$ or from I_1 if $t_j = \texttt{'1'}$.

5.2 Verification on Reachable and Unreachable Targets

Table 3 summarizes the exploit verification results recorded in `attack_demo` (`submission.json`). Because a permutation $\pi \in S_n$ bijectively reorders indices without creating or destroying ones, $\sum_{j=0}^{n-1} s_{\pi(j)} = \sum_{i=0}^{n-1} s_i$. Thus, a target fingerprint t is reachable from s if and only if $\mathrm{wt}_H(s) = \mathrm{wt}_H(t)$.

Exploit Field	**Reachable Case ($\mathbf{wt}_H = 2 \to 2$)**	**Unreachable Case ($\mathbf{wt}_H = 2 \to 3$)**
Source Fingerprint (s)	`1100` (2 ones, 2 zeroes)	`1100` (2 ones, 2 zeroes)
Target Fingerprint (t)	`0101` (2 ones, 2 zeroes)	`1110` (3 ones, 1 zero)
Crafted `compatibility_map`	`(2, 0, 3, 1)`	`None`
Canonical Fingerprint	`1100`	N/A
Adapter Reported Fingerprint	`0101`	N/A
Outcome	`exploit_succeeded = True`	`is_impossible = True`

Table 3: Exploit verification via `service.simulate_adapter` (`attack_demo`).

6 Research Extensions

6.1 Extension 1: Exhaustive 16×24 Permutation Orbit-Stabilizer Analysis

We exhaustively evaluated all $2^4 = 16$ four-bit fingerprints across all $4! = 24$ permutations in S_4 (384 combinations). By the Orbit-Stabilizer Theorem, for a bitstring $x \in \{0,1\}^4$ of Hamming weight $w = \mathrm{wt}_H(x)$:

- The number of permutations that **preserve** x (stabilizer size) is $|\mathrm{Stab}(x)| = w!(4-w)!$, which permutes the w ones among themselves and the $4-w$ zeroes among themselves.
- The number of permutations that **alter** x is $24 - w!(4-w)!$.
- The number of **distinct reachable fingerprints** (orbit size) is $|\mathcal{O}(x)| = \frac{24}{w!(4-w)!} = \binom{4}{w}$.

Hamming Weight w	**Example Fingerprints**	**Total Perms**	**Preserving $\lvert\mathbf{Stab}(x)\rvert$**	**Altering Perms**	**Reachable Orbit $\binom{4}{w}$**
$w = 0$	`0000` (1 case)	24	$0!4! = 24$	0	1
$w = 1$	`0001, 0010, 0100, 1000` (4 cases)	24	$1!3! = 6$	18	4
$w = 2$	`0011, 0101, 0110, 1001, 1010, 1100` (6 cases)	24	$2!2! = 4$	20	6
$w = 3$	`0111, 1011, 1101, 1110` (4 cases)	24	$3!1! = 6$	18	4
$w = 4$	`1111` (1 case)	24	$4!0! = 24$	0	1

Table 4: Exhaustive 16×24 analysis grouped by Hamming weight w (`summary_by_hamming_weight`).

6.2 Extension 2: n-Bit Generalization & Scalability Analysis ($n = 16$)

We generalized both the canonical decoder (`secure_decode_n_bits`) and the linear-time permutation exploit (`craft_compatibility_map_n_bits`) to arbitrary bitlength n and validated them on $n = 16$ bits (`src_16` = `'1011001011010001'`, `tgt_16` = `'0110100110101100'`, both with Hamming weight 8):

- **Exploit Scalability:** Exhaustive permutation search scales as $\mathcal{O}(n!)$, which at $n = 16$ requires searching $16! = 20,922,789,888,000$ permutations and is intractable beyond $n > 12$. By contrast, our index-queue matching algorithm constructs a valid 16-bit permutation (`valid_permutation_found = True`) in $\mathcal{O}(n)$ time and space.

- **Quantum vs. Classical Simulation Scaling:** The Bernstein-Vazirani circuit depth scales as $\mathcal{O}(n)$ CX gates and extracts the n-bit secret in 1 quantum query (vs. $\mathcal{O}(n)$ classical queries), whereas classical statevector simulation scales exponentially as $\mathcal{O}(2^n)$ memory and time.

6.3 Extension 3: Blind Metamorphic Invariance Suite

To detect order-dependent adapter bugs without knowing the secret registry fingerprint in advance, we tested the metamorphic relation:

$$\text{Decision_Key}(\text{Circuit}, \text{Metadata}_A) \stackrel{?}{=} \text{Decision_Key}(\text{Circuit}, \text{Metadata}_B)$$

Across three unknown probe circuits (`'1010'`, `'1100'`, `'1000'`) evaluated under $\text{Metadata}_A = [0, 1, 2, 3]$ and $\text{Metadata}_B = [1, 0, 3, 2]$, the legacy adapter exhibited order-dependent violations on $2/3$ probes (`legacy_violations_count = 2`), whereas `secure_decode_n_bits` had 0 violations (`secure_violations_count = 0`).

7 Claim Sheet & Oral-Defense Summary

Table 5 presents our structured claim sheet exported to `submission.json`.

Claim	Experimental Evidence	Trusted Assumption	Limitation
1. The fault is NOT quantum noise.	Phase A & B (`results`, `localization_evidence`): Raw backend counts are 100% deterministic ($512/512$ shots on a single bitstring, e.g., `{'0101': 512}` for `MIG-DELTA`) and match measurement-free `Statevector` probabilities ($p = 1.0$).	`BasicSimulator` executes ideal unitary gates without decoherence or readout errors.	On real NISQ hardware, gate and readout noise distribute shots across multiple keys, requiring a statistical threshold around `dominant_key(counts)`.
2. The circuit DOES encode the registry value.	Phase B (`circuit.draw`, `Statevector`, `canonical_bits_from_counts`): Inspecting the BV CX gates and reversing Qiskit's little-endian key yields `'1010'` for `MIG-DELTA` and `'1100'` for `INCIDENT-7`, matching `registry_fingerprint`.	Qubit index i maps to canonical slot i ($q_0 \dots q_3$) and `build_bv_probe` is untampered.	Verifies the demo oracle matches the registry, but cannot detect if an external actor replaces the quantum circuit itself with a spoofed oracle.
3. Metadata CAN alter the legacy decision (and CANNOT alter `secure_decision`).	Phase B, C & E (`controlled_experiment`, `minimal_reproducer`, `attack_demo`): Holding the circuit constant (`'1000'` or `'1010'`) while changing `compatibility_map` from `[0,1,2,3]` to `[1,0,2,3]` flips `MATCH` to `MISMATCH`.	Legacy service calls `compatibility_decode` on untrusted manifest metadata prior to comparison.	Hamming-weight invariant: permutation tampering can only reach target strings with the exact same number of 0s and 1s.
4. The mitigation generalizes.	Phase D & Extensions (`defense_results`, `exhaustive_analysis`, `scalability_analysis`): `secure_decode` and `secure_decision` pass all $13/13$ tests, all 16×24 4-bit cases, $n = 16$ scaling, and metamorphic tests.	Registry fingerprints are stored in canonical $q_0 \dots q_{n-1}$ order and raw counts come from genuine probe execution.	Secures the classical decoding boundary against metadata tampering, but does not prevent memory corruption of raw `counts` or `registry_fingerprint`.

Table 5: Structured claim sheet from `01_trust_no_input.ipynb` and `submission.json`.

Oral-Defense Quick Reference

- **Why is the dominant Qiskit count key not automatically the canonical logical fingerprint?** Because Qiskit formats classical registers in big-endian order ($c_{n-1} \dots c_0$, leftmost bit highest), whereas the contract encodes slot i on qubit q_i ($q_0 \dots q_{n-1}$). Reversing the string (`dominant_key(counts)[::-1]`) is required to align bit index i with qubit q_i.
- **Why is validating that `compatibility_map` is a valid permutation (`_validate_map`) insufficient?** The legacy service *already* validates that `compatibility_map` is a valid permutation of $0 \dots n-1$. The vulnerability exists because a valid non-identity permutation still reorders bit positions across the security boundary, enabling both denial-of-service (`MIG-DELTA`) and same-Hamming-weight spoofing (`INCIDENT-7`).
- **Where should legacy presentation conversion occur?** Strictly in a read-only UI/display presenter *after* `secure_decision(counts, registry_fingerprint)` has already computed and recorded the immutable security verdict from the canonical representation.